Interfaces - Pattern - Networks

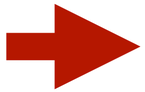
2026 - ATIAM

00 - Interfaces

Definitions

Interaction

Action réciproque de deux ou plusieurs objets, de deux ou plusieurs phénomènes. (TLFi)



Interface

Plan ou surface de discontinuité formant une frontière commune à deux domaines aux propriétés différentes et unis par des rapports d'échanges et d'interaction réciproques. (Larousse)

INFO: Jonction entre deux matériels ou logiciels leur permettant d'échanger des informations par l'adoption de règles communes physiques ou logiques (TLFi)

Physical Interfaces





Graphical User Interfaces (GUI)

World Wide Web

The WorldWideWeb (W3) is a wide-area hypermedia information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an executive summary of the project, Mailing lists , Policy , November's W3

What's out there?
: Pointers to the world's online information, subjects , W3 servers, etc.

Help
: on the browser you are using

Software Products
: A list of W3 project components and their current state. (e.g. Line Mode ,X11 Viola , NeXTStep , Servers , Tools , Mail robot , Library)

Technical
: Details of protocols, formats, program internals etc

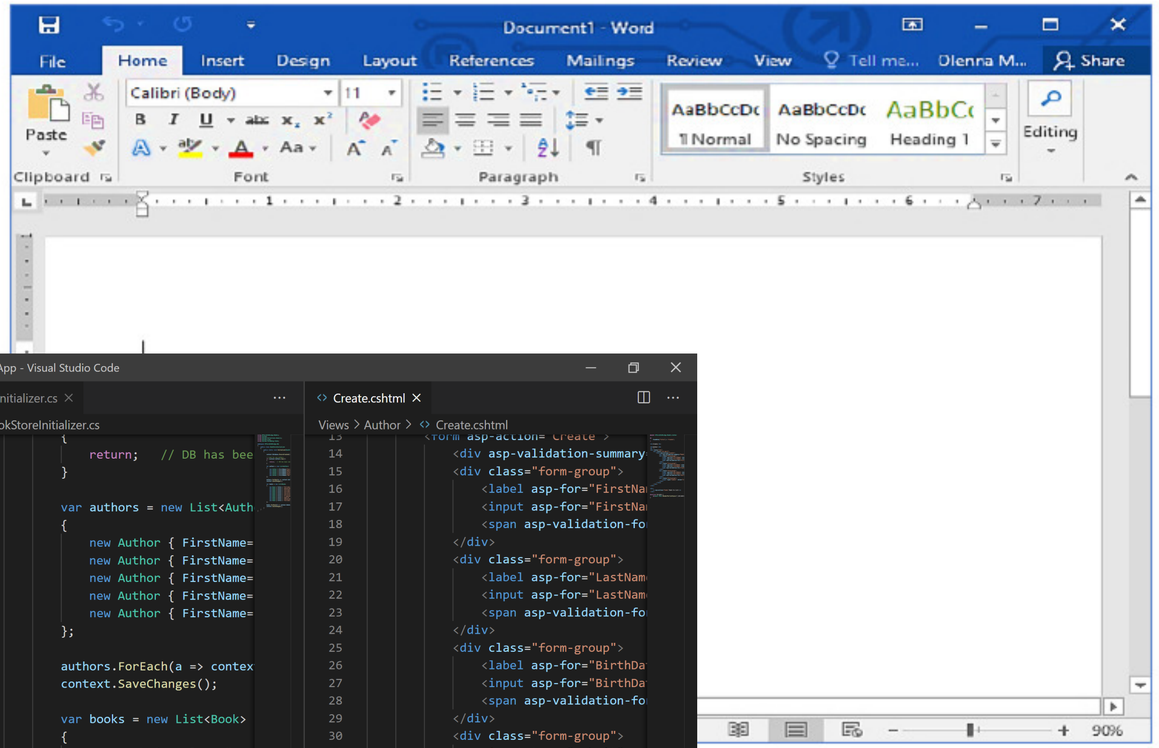
Bibliography
: Paper documentation on W3 and references.

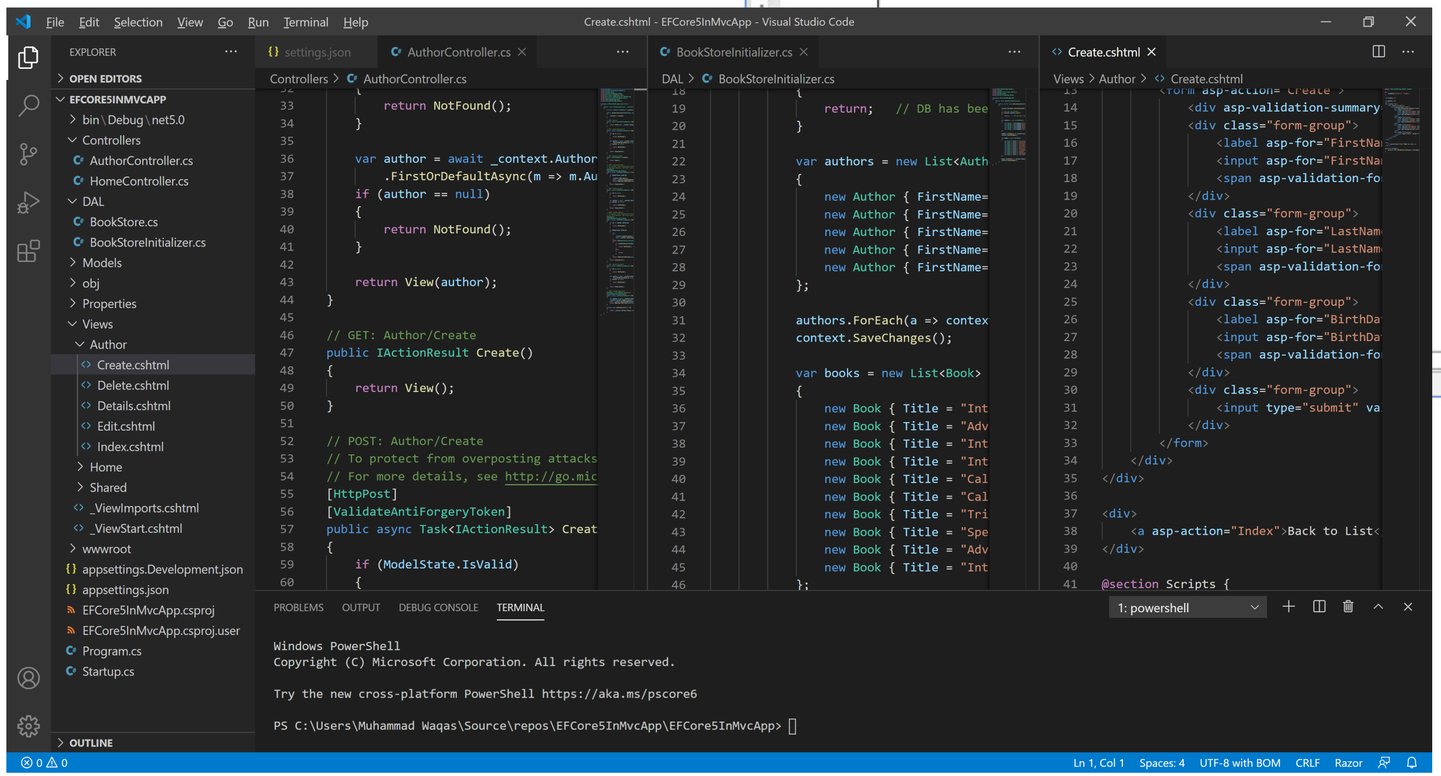
People
: A list of some people involved in the project.

History
: A summary of the history of the project.

How can I help ?
: If you would like to support the web..

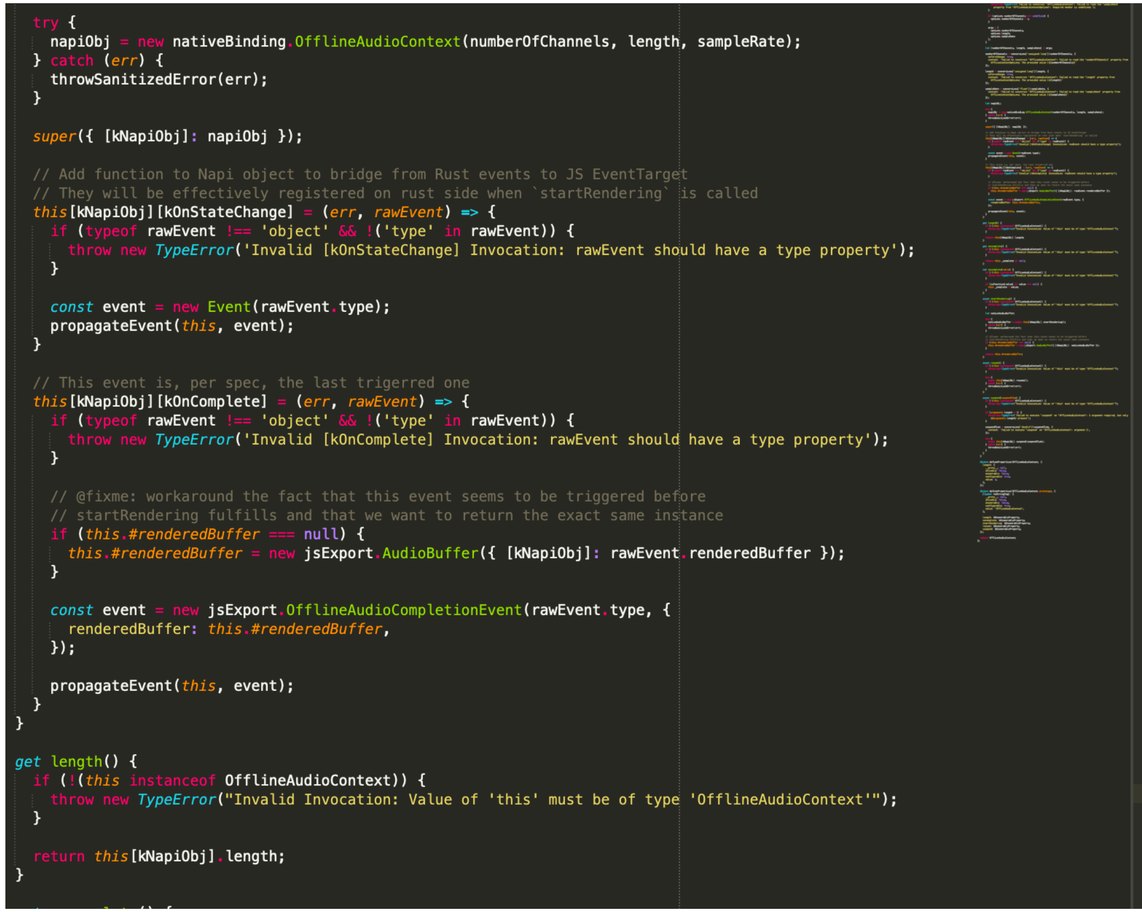
Getting code
: Getting the code by anonymous FTP , etc.





General Purpose Programming Languages

Scripting language, e.g. JavaScript



```
  try {
    napiObj = new nativeBinding.OfflineAudioContext(numberOfChannels, length, sampleRate);
  } catch (err) {
    throwSanitizedError(err);
  }

  super({ [kNapiObj]: napiObj });

  // Add function to Napi object to bridge from Rust events to JS EventTarget
  // They will be effectively registered on rust side when `startRendering` is called
  this[kNapiObj][kOnStateChange] = (err, rawEvent) => {
    if (typeof rawEvent !== 'object' && !('type' in rawEvent)) {
      throw new TypeError('Invalid [kOnStateChange] Invocation: rawEvent should have a type property');
    }

    const event = new Event(rawEvent.type);
    propagateEvent(this, event);
  }

  // This event is, per spec, the last trigerred one
  this[kNapiObj][kOnComplete] = (err, rawEvent) => {
    if (typeof rawEvent !== 'object' && !('type' in rawEvent)) {
      throw new TypeError('Invalid [kOnComplete] Invocation: rawEvent should have a type property');
    }

    // @fixme: workaround the fact that this event seems to be triggered before
    // startRendering fulfills and that we want to return the exact same instance
    if (this.#renderedBuffer === null) {
      this.#renderedBuffer = new jsExport.AudioBuffer({ [kNapiObj]: rawEvent.renderedBuffer });
    }

    const event = new jsExport.OfflineAudioCompletionEvent(rawEvent.type, {
      renderedBuffer: this.#renderedBuffer,
    });

    propagateEvent(this, event);
  }
}

get length() {
  if (!(this instanceof OfflineAudioContext)) {
    throw new TypeError("Invalid Invocation: Value of 'this' must be of type 'OfflineAudioContext'");
  }

  return this[kNapiObj].length;
}
```

System language, e.g. Rust

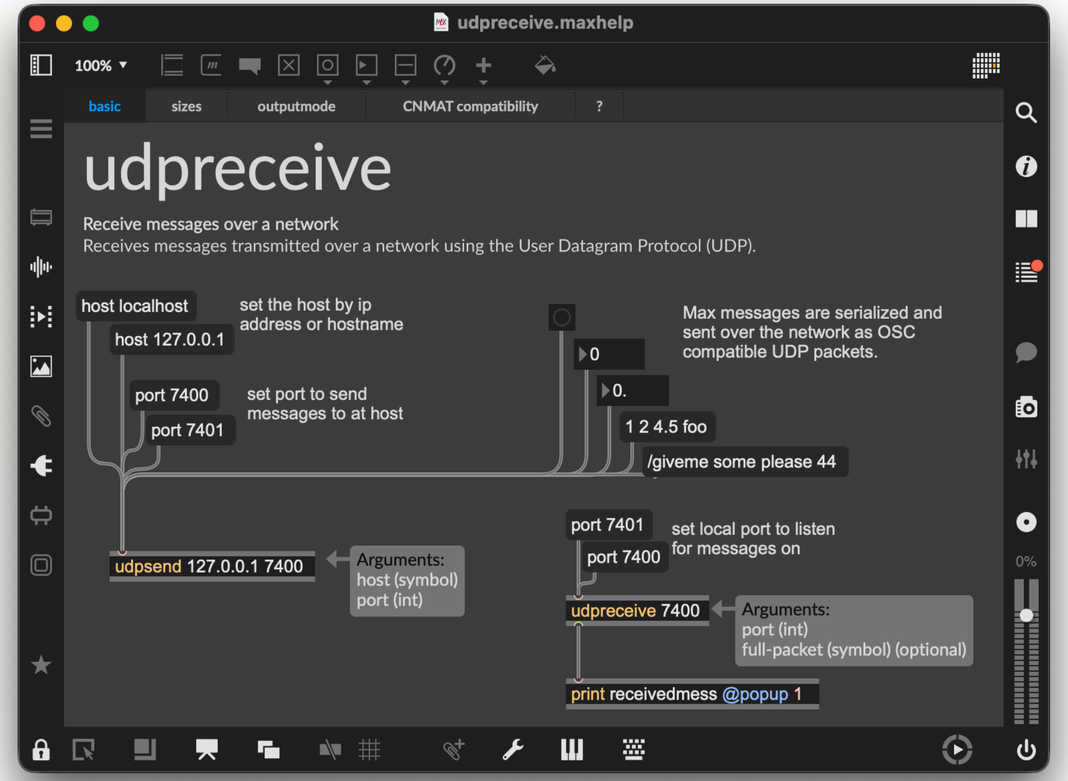
```
130
131  #[js_function]
132  fn start_rendering(ctx: CallContext) -> Result<JsObject> {
133      let js_this = ctx.this_unchecked::<JsObject>();
134      let napi_context = ctx.env.unwrap::<NapiOfflineAudioContext>(&js_this)?;
135      let context = napi_context.unwrap();
136
137      let k_onstatechange = get_symbol_for(ctx.env, "node-web-audio-api:onstatechange");
138      let statechange_cb = js_this.get_property(k_onstatechange).unwrap();
139      let mut statechange_tsfn = ctx.env.create_threadsafe_function(
140          &statechange_cb,
141          0,
142          |ctx: ThreadSafeCallContext<Event>| {
143              let mut event = ctx.env.create_object()?;
144              let event_type = ctx.env.create_string(ctx.value.type_)?;
145              event.set_named_property("type", event_type)?;
146
147              Ok(vec![event])
148          },
149      )?;
150
151      let k_oncomplete = get_symbol_for(ctx.env, "node-web-audio-api:oncomplete");
152      let complete_cb = js_this.get_property(k_oncomplete).unwrap();
153      let mut complete_tsfn = ctx.env.create_threadsafe_function(
154          &complete_cb,
155          0,
156          |ctx: ThreadSafeCallContext<OfflineAudioCompletionEvent>| {
157              let raw_event = ctx.value;
158              let mut event = ctx.env.create_object()?;
159
160              let event_type = ctx.env.create_string("complete")?;
161              event.set_named_property("type", event_type)?;
162
163              // @fixme: this event is propagated before `startRedering` fulfills
164              // which is probaly wrong, so let's propagate the JS audio buffer
165              // and let JS handle the race condition
166              let ctor = get_class_ctor(&ctx.env, "AudioBuffer")?;
167              let js_audio_buffer = ctor.new_instance(&[ctx.env.get_null()?])?;
168              let napi_audio_buffer = ctx.env.unwrap::<NapiAudioBuffer>(&js_audio_buffer)?;
169              napi_audio_buffer.insert(raw_event.rendered_buffer);
170
171              event.set_named_property("renderedBuffer", js_audio_buffer)?;
172
173              Ok(vec![event])
174          },
175      )?;
176
177      // unref tsfn so they do not prevent the process to exit
```

Domain Specific Languages (DSL)

SQL - interact with database

```
4    SELECT
5      user_id
6    FROM
7      USERS_BEHAVIORS_CUSTOMEVENT_SHARED
8    WHERE
9      name='MY_CUSTOM_EVENT'
10   GROUP BY 1
11   HAVING COUNT(id) > 0;
```

Audio - *(DSL or GUI ?)*



Application Programming Interface (API)

```
/**
 * A Communication Channel
 */
class Pubsub {
  /** construct a new PubSub instance */
  constructor() {
    // ...
  }

  /** Emit a message on a given channel */
  publish(channel: String, ...args: Any[]) {
    // ...
  }

  /** Emit a message on a given channel */
  subscribe(channel: String, callback: Function) {
    // ...
  }

  /** Some internal method */
  private #doSomething() {

  }
}
```

An application programming interface (API) is a connection between computers or between computer programs or between program components.

It is a type of software interface, offering a service or a contract to other pieces of software.

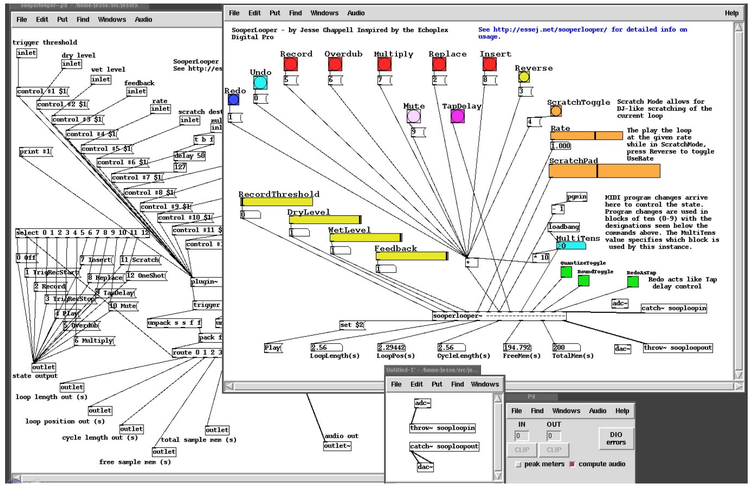
Adapted from Wikipedia

So, what is an interface… ? (In CS)

- Something that defines a boundary between 2 (or more) (sub-)systems
- An interface often hides another lower-level interface
- Open possibilities but close other ones

i.e. GUI are often easier or faster to interact with but can be less powerful than lower level interfaces (such as code or DSL)





Implications

- Interfaces result from choices (or mistakes) made by people
- Cannot be considered as scientific ***true / false questions*** but rather as ***good / bad solutions***

Example : https://brainfuck.net/

"Brainfuck is Turing-complete, meaning it can theoretically compute anything that any other programming language can. It's just... not very practical. And that's the point."

Design as a discipline

A design is the concept or proposal for an object, process, or system. The word design refers to something that is or has been intentionally created by a thinking agent, and is sometimes used to refer to the inherent nature of something – its design.

Wikipedia

Design as a discipline

Several approaches:

- positivist: *The Sciences of the Artificial*, Herbert A. Simon, 1969 [link]

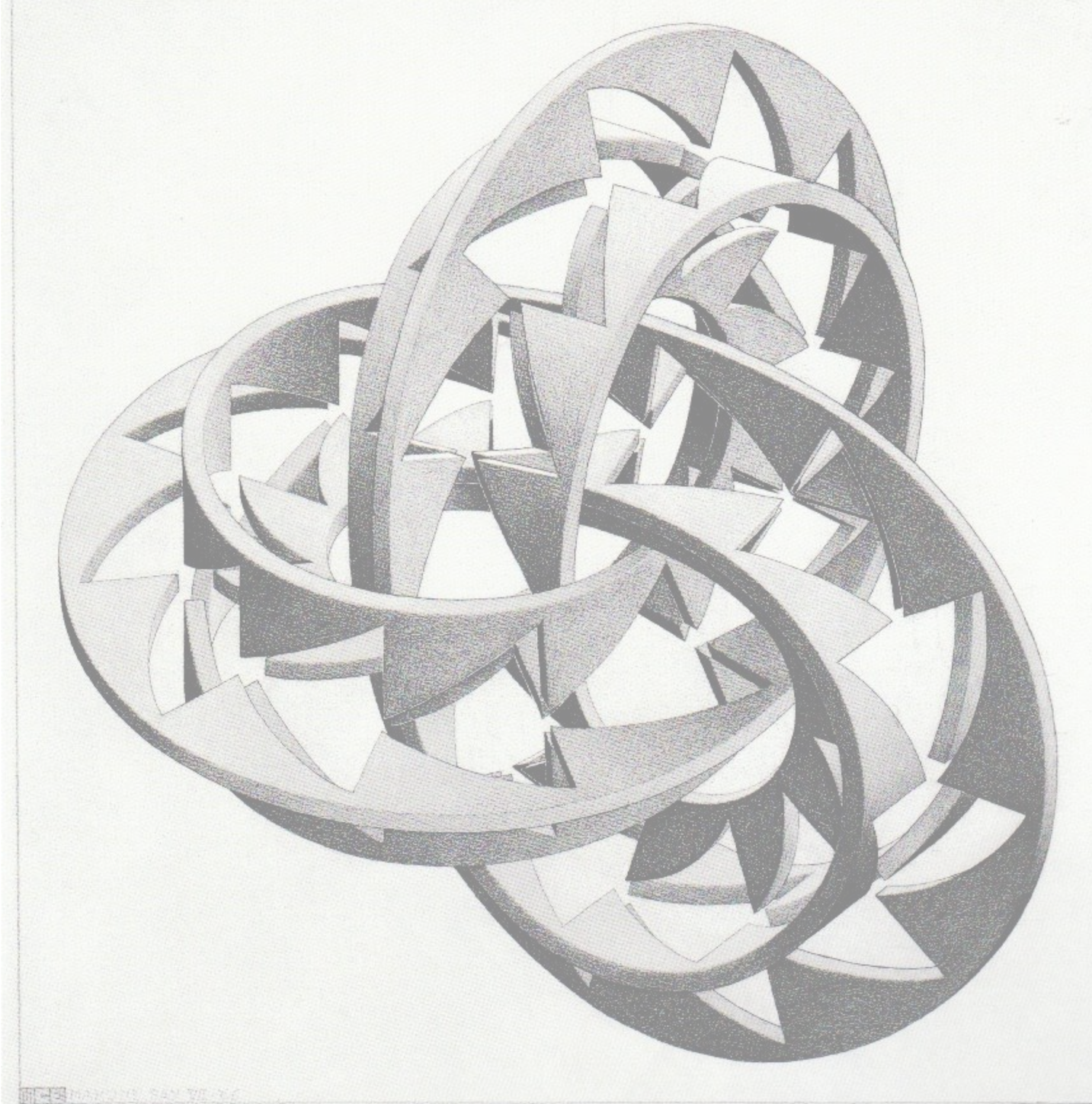
- non-positivist: *Dilemmas in a General Theory of Planning,* Rittel & Webber, 1973 [link]

Need to take into account multiple (and sometimes conflicting) dimensions, e.g. humans, habits, society, etc.

Examples: AZERTY keyboard, C-like languages, etc…

Notion of ***strong concepts*** compared to ***proofs***

01 - Patterns



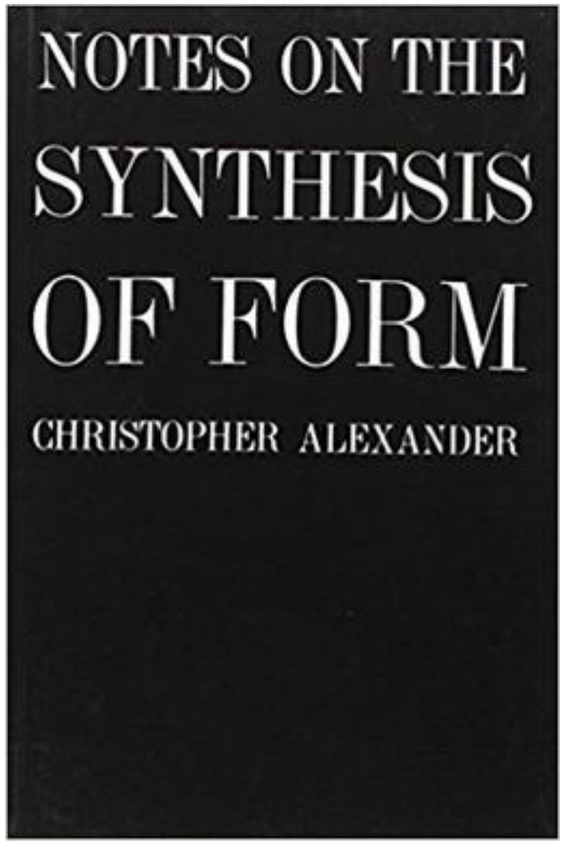
Design Patterns

A software design pattern describes a reusable solution to a commonly needed behavior in software. A design pattern is not a rigid structure to be copied directly into source code. Rather, it is a description of and a template for solving a particular type of problem that can be used in many different contexts

Wikipedia

Design Patterns - History

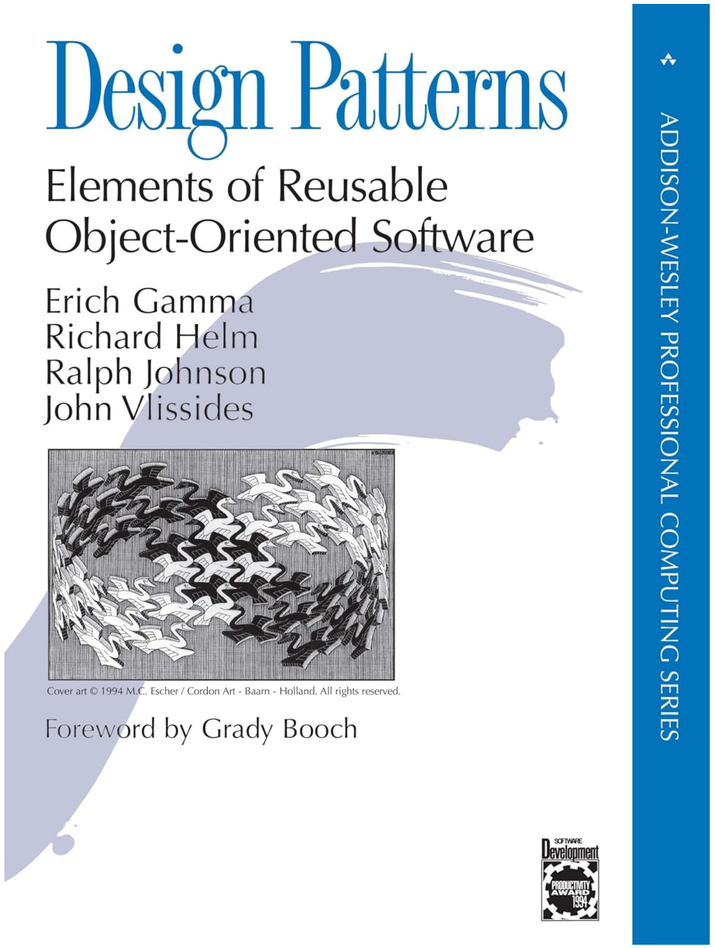
Notes on the Synthesis of Form, C. Alexander, 1964 [link]



- Domain of architecture
- Adaptation of the *form* to a *context*
- Patterns describe common articulation between form and context

Design Patterns - Computer Science

Design Patterns: Elements of Reusable Object-Oriented Software, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, 1994 [link]



Creational patterns
patterns that create objects, rather than having to instantiate objects directly. This gives the program more flexibility in deciding which objects need to be created for a given case.

Structural patterns
concern class and object composition. They use inheritance to compose interfaces and define ways to compose objects to obtain new functionality.

Behavioral patterns
patterns are specifically concerned with communication between objects.

Design Patterns - In practice

The Singleton Pattern

Restricts the instantiation of a class to a singular instance.

https://github.com/b-ma/course-interface-patterns-network/tree/main/01-singleton

Publish / Subscribe pattern (PubSub)

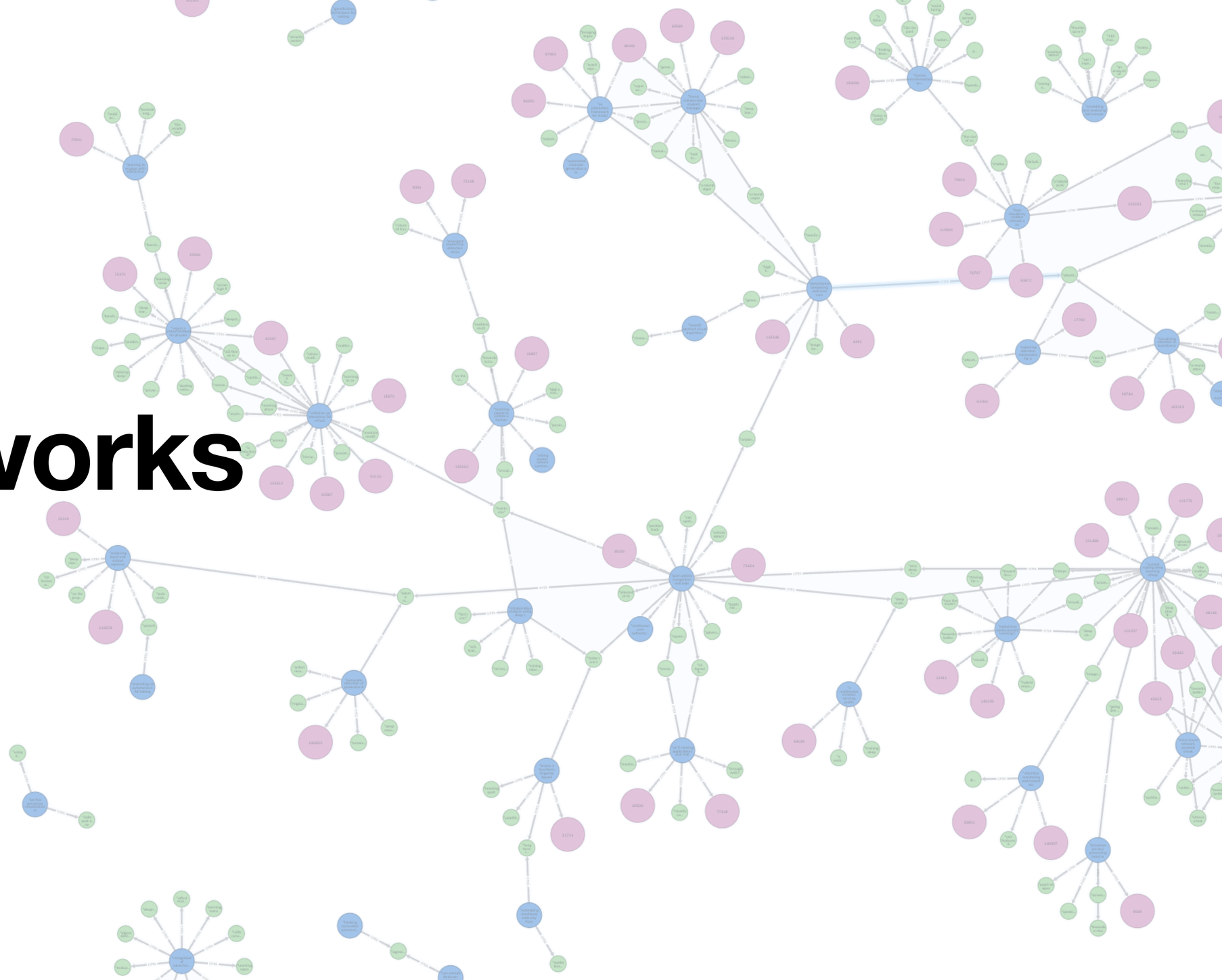
In software architecture, the publish–subscribe pattern (pub/sub) is a messaging pattern in which message senders, called publishers, categorize messages into classes (or topics), and send them without needing to know which components will receive them. Message recipients, called subscribers, express interest in one or more classes and only receive messages in those classes, without needing to know the identity of the publishers.

- Decouples components that produce messages from those that consume them
- Possibly asynchronous
- Many-to-many communication

PubSub - In practice

https://github.com/b-ma/course-interface-patterns-network/tree/main/02-pubsub

02 - Networks



Question

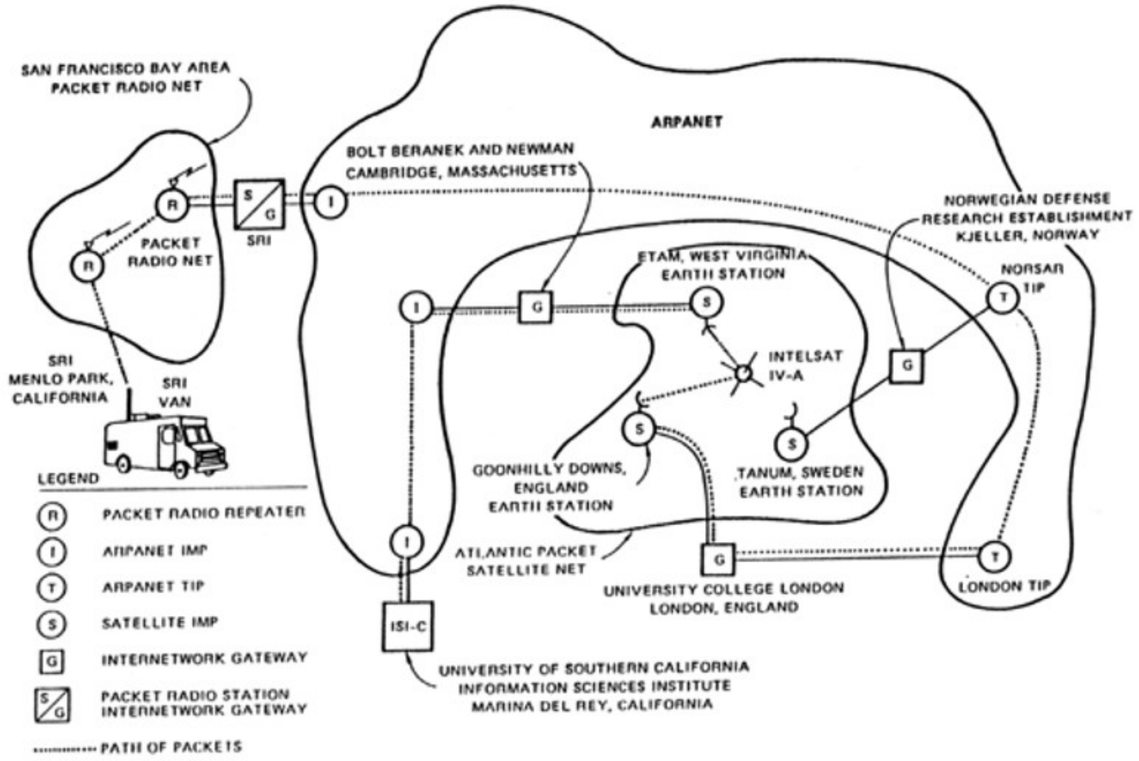
To how many networks your cellphone is connected to ?

For information

Licklider, Joseph CR. "Man-computer symbiosis." IRE transactions on human factors in electronics 1 (1960): 4-11. [link]

Internet Protocol (IP)

Network Protocol (OSI layer 3)

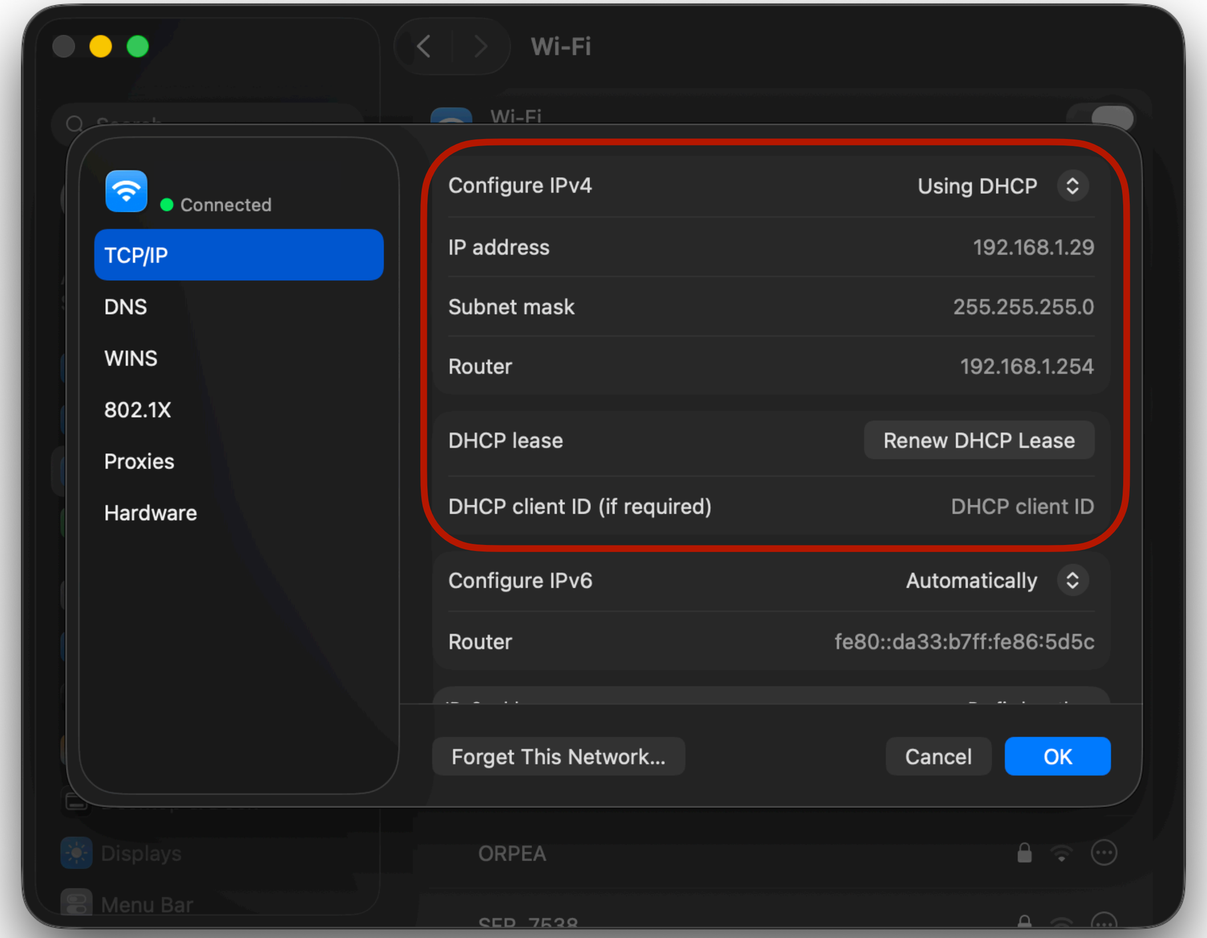


Responsibility

- addressing host interfaces
- encapsulating data into datagrams
- routing datagrams from a source host interface to a destination host interface

Internet Protocol (IP)

Network Protocol (OSI layer 3)



- IP Address
- Fix IP or DHCP ?
- Router
- Subnet Mask

User Datagram Protocol (UDP)

Transport Protocol (OSI layer 4)

UDP is a connectionless protocol, meaning that messages are sent without negotiating a connection and that UDP does not keep track of what it has sent.

No guarantee on:

- delivery - *messages can be lost*
- ordering - *messages can be received in wrong order*
- duplicate protection - *messages can be received multiple times*

But, simple and fast

Transmission Control Protocol (TCP)

Transport Protocol (OSI layer 4)

Provides reliable, ordered, and error-checked delivery of a stream of octets between applications running on hosts communicating via an IP network.

Guarantee on:

- delivery - *messages cannot be lost*
- ordering - *messages are received in right order*
- duplicate protection - *messages cannot be received multiple times*

But, simple and fast

UDP - In practice

https://github.com/b-ma/course-interface-patterns-network/tree/main/03-udp

03 - Communication Protocols

Open Sound Control (OSC)

Communication Protocol

Protocol for controlling sound synthesizers, computers, and other multimedia devices over the network for purposes such as musical performance or show control.

Specification: https://opensoundcontrol.stanford.edu/spec-1_0.html

Basically a PubSub (typically) implemented over the UDP transport protocol

OSC - In Practice

https://github.com/b-ma/course-interface-patterns-network/tree/main/04-osc

WebSocket

Communication Protocol

WebSocket is a computer communications protocol, providing a bidirectional communication channel over a single Transmission Control Protocol (TCP) connection.

Specification: https://www.rfc-editor.org/info/rfc6455/
(Internet Engineering Task Force (IETF)

WebSockets - In Practice

https://soundworks.dev/misc/websockets-101.html

Thanks

2026 - ATIAM